

LabPath

Practice platform for AI engineering

TRAIN HERE. HIRE A TUTOR IF YOU WANT TO BE TAUGHT.



VERSION

v1.0

DATE

25 August 2026

OWNER

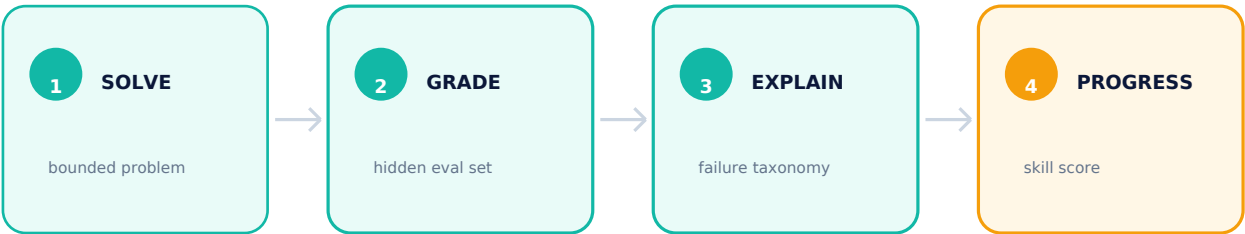
Yonatan | Bina / CyberProAI

STATUS

Draft for engineering review

Executive product map

LabPath is the gym for AI engineers: learners solve bounded, real problems; the platform grades behaviour against hidden evaluation sets; failures become targeted feedback; verified performance becomes a credible signal. Teaching remains human and on demand.



Dimension	Decision
Product	Practice platform with objective grading - no lessons
POC	10 exercises across RAG, LLM Evaluation, and Prompt Injection & Guardrails
Core moat	Versioned hidden eval sets, deterministic harnesses, calibrated judges, failure taxonomy
Primary metric	Activation: first-session submission; feedback helpfulness
Phase 0	6-8 weeks; no arbitrary code execution; constrained prompts/config/DSL only
Strategic path	Public beta -> competition -> verified credibility -> tutor marketplace -> teams/institutions

Document metadata	Value
Document	Product & Technical Specification v1.0
Owner	Yonatan - Bina / CyberProAI
Date / status	25 August 2026 - Draft for engineering review
Prototype	learningapp-web.vercel.app - LabPath mockup
Audience	Engineering, product, design

NORTH STAR

Reps and a score

Every feature must strengthen one of four loops: more practice, more trustworthy measurement, more useful failure feedback, or more credible proof.

Contents

Executive product map	2
Contents	3
How to read this document	5
1. Positioning	7
One-line positioning	7
What LabPath is not	7
Why this is defensible	7
2. The problem being solved	8
The product's flywheel	8
3. Product principles	9
4. Users	10
4.5 Buyer personas - later phases	10
5. Content model	11
5.1 Exercise-as-code	11
5.2 Skill graph	11
6. Simulator catalogue	12
6.1 Why RAG, Evaluation, and Guardrails form the POC	12
7. Grading & feedback engine	13
7.1 Grading pipeline	13
7.2 Three grader classes	13
7.3 Feedback quality bar	14
7.4 Instrumentation panel	14
7.5 Grader hardening	14
7.6 How verdict stability is engineered	15
7.7 Never gate on wall-clock latency	15
8. Progression, profile and credibility	16
9. Tutor layer - Preply model	17
9.1 Thesis	17
9.2 Marketplace shape	17
9.3 Build only the foundations now	17
10. Monetization	18
11. Target architecture	19
Key choices	19
12. Non-functional requirements	20
12.1 Reproducibility	20
12.2 Execution safety	20
12.3 Cost and abuse control	20
12.4 Anti-cheat	21
12.5 Performance	21
12.6 Accessibility and i18n	21

13. Success metrics	22
Phase overview	24
Phase 0 - The POC	25
0.1 Objective	25
0.2 Scope	25
0.3 POC data model	26
0.4 POC API surface	26
0.5 The ten exercises	27
RAG simulator - shared environment	27
LLM Evaluation simulator - shared environment	29
Prompt Injection & Guardrails simulator	30
0.6 Coverage check	32
0.7 UI scope - four screens	32
0.8 Phase 0 acceptance criteria	33
0.9 Week-one engineering blockers	34
0.9.1 Assertion DSL	34
0.9.2 Slice Spec - E3 only	35
0.9.3 Human labelling protocol	35
0.9.4 exercise.yaml schema and ingest	36
0.10 Per-exercise cost budgets	37
0.11 Team and sequencing	38
0.12 Phase 0 risks	39
Phase 1 - Public beta +10 weeks	40
Phase 2 - Depth & competition +12 weeks	40
Phase 3 - Credibility +10 weeks	41
Phase 4 - Tutor marketplace +12 weeks	41
Phase 5 - Teams, institutions and Bina +12 weeks	41
Open decisions	42
Appendix A - Glossary	43
Appendix B - References	43

How to read this document

Part 1 describes the end-state product - what LabPath becomes, and the decisions that must be true from day one so the end state remains reachable.

Part 2 is the build sequence. Phase 0 is a ten-exercise proof of concept across three simulators, specified deeply enough for engineering to begin immediately. Phases 1-5 are scoped now and re-specified as each approaches.

DECISION

Expensive to reverse

Sections marked DECISION establish architecture, product, or integrity constraints that should not drift through implementation.

OPEN

A call is still required

Sections marked OPEN identify a decision that must be resolved before the relevant phase starts.



1

PART 1

THE END GOAL

Positioning, users, content model, grading integrity, progression, marketplace, monetization and architecture.

1. Positioning

LeetCode made algorithm practice a habit and a credential. Preply made language tutoring a marketplace. AI engineering has neither.

LabPath is a place to practice AI engineering against real, graded problems - with no lessons. If you want to be taught, you hire a tutor from inside the platform.

The deliberate omission is the product. Incumbents sell content: courses, videos, tracks, and completion certificates. Content ages quickly and teaches recognition more readily than capability. LabPath sells **reps and a score**. The teaching layer is human, on demand, and contextual to the learner's actual failure.

One-line positioning

The gym for AI engineers. Sets, reps, and a number that goes up. Coaches available at the desk.

What LabPath is not

- Not a course platform. No video lessons, curriculum, or 'Module 3 of 12'.
- Not a certification body at first; verified assessments arrive in Phase 3.
- Not a hosted AI-app builder. Learners solve bounded problems rather than shipping products.
- Not a general coding site. The unit of work is an AI system's behaviour, not an algorithm's complexity.

Why this is defensible

The moat is not the exercise list; that can be copied. The moat is the **grading substrate**: hidden evaluation sets, deterministic scoring harnesses, and accumulated calibration data describing how thousands of engineers fail each problem. That data improves feedback, powers the skill graph, and eventually underwrites the credential. It also gives a tutor useful context in the first minutes of a session.

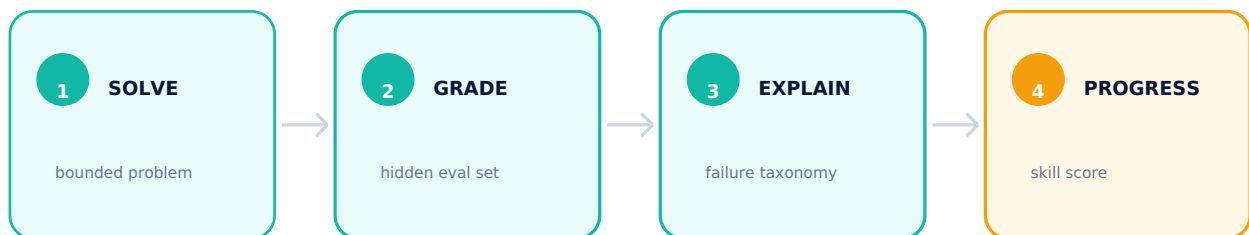
2. The problem being solved

AI engineering has an unusually wide gap between reading about a technique and being able to operate it reliably in production.

1. **Outputs are non-deterministic.** Evaluation must be learned by watching a pipeline fail across enough cases, not by reading a single successful example.
2. **Failure modes are invisible without instrumentation.** A RAG system that answers 8 of 10 questions well can feel finished; the 2 hallucinated citations are the real job.
3. **The stack is unstable but the skills are durable.** Chunking, retrieval quality, eval design, judge calibration, injection defence, and cost/latency tradeoffs survive model and framework churn.
4. **Capability is hard to prove.** Hiring managers currently proxy with 'have you shipped a RAG app?' There is no LeetCode-equivalent signal.

LabPath answers all four with bounded problems, hidden eval sets, visible instrumentation, framework-agnostic skills, and a score that means something.

The product's flywheel



3. Product principles

These are constraints on every future decision, not aspirations.

#	Principle	Consequence
P1	Every exercise has objective pass/fail.	Metric, threshold, hidden test set; never 'AI says it looks good'.
P2	Deterministic checks before LLM judgement.	Judges only where semantics truly require them; calibrate against human labels.
P3	Feedback is the product.	Show which cases failed, why, and the class of mistake - never a grade alone.
P4	Hidden test sets, always.	Public sample plus held-out scoring prevents overfitting and copied answers.
P5	Framework-agnostic.	Grade behaviour and metrics, not whether a named framework was used.
P6	Bounded runtime and bounded cost.	Server-enforced token, time, and currency budgets on every attempt.
P7	No lessons.	Hints and explanations of the learner's own failure are allowed; pre-taught curriculum is not.
P8	Stable verdicts.	Same submission re-graded returns the same pass/fail with $\geq 99\%$ probability.

DECISION

P2 is the most important line

Judge-only scoring is the wrong shortcut for training. The score must move when the learner's work moves. Deterministic facts - recall@5, canary leakage, forbidden tool calls - are scored first; calibrated judges cover only irreducibly semantic questions.

OPEN 07

Verify competitor scoring first-hand

Spend one hour testing Lakera's Agent Breaker with repeated identical inputs. Record how scoring behaves before any comparative claim is circulated externally.

4. Users

Priority	Persona	Context	Job to be done
Primary	The converting engineer	Backend/full-stack/data engineer, 2-8 years; has shipped an API-based chatbot and now owns an AI feature.	Reps, instrumentation, a scoreboard; ~€15-25/mo and €30-80/hr tutor willingness.
Secondary	The interview candidate	High-intensity, short-lived user interviewing for applied AI roles.	Curated interview problems and a profile signal.
Secondary	The ML person moving up-stack	Strong model knowledge; weaker on production evals, guardrails, agents, and cost.	Depth at medium/hard difficulty; low churn.
Tertiary	The security engineer	Enters through injection and guardrails; overlaps with Bina's cyber audience.	Viral red-team entry point.

4.5 Buyer personas - later phases

- **Engineering manager** buying seats and a team dashboard.
- **University or government programme** buying cohorts through Bina's existing motion.
- **Tutor** supplying instruction to stuck learners; vetted by solving LabPath exercises.

5. Content model

DECISION

Five types, one grading substrate

Every content item is one **Exercise** database record with a **type** field. Types differ in the harness they invoke, not in the data model. This decision is what lets the catalogue grow without a rewrite.

Type	Duration	Learner produces	Graded by
Drill	2-5 min	Single prompt, config value, or short answer	Deterministic assertion
Exercise	10-30 min	Configuration, prompt, or small code block	Hidden set + metric threshold
Lab	45-120 min	Working component: retriever, eval suite, guardrail	Multi-metric scorecard
Simulation	20-90 min	Interaction with adversarial/dynamic environment	Objective event detection
Scenario	2-4 hrs	End-to-end system under constraints	Full scorecard + optional tutor review

5.1 Exercise-as-code

Exercises live as **version-controlled YAML and asset files in a Git repository**, not in a CMS or admin UI. They need review, CI, rollback, and diffable history. An authoring UI may later write to the repository through pull requests.

```
/exercises
/rag-001-chunk-it-right
exercise.yaml
brief.md
/corpus
eval_public.jsonl
eval_hidden.jsonl
labels.jsonl
grader.py
/solution
hints.md
```

5.2 Skill graph

Each exercise maps to controlled **skill nodes** with difficulty weights. The graph is a DAG, for example **chunking** -> **retrieval quality** -> **reranking**. It drives learner radar profiles, next-exercise recommendations, tutor matching, and later team gap analysis. Start with roughly 30 nodes; exercise authors may not invent free-form tags.

6. Simulator catalogue

A simulator is an **environment**, not an exercise. One simulator hosts many exercises at many difficulty levels.

#	Simulator	What it trains	Build	Breadth	Phase
1	RAG	Chunking, embeddings, retrieval, reranking, grounding, citations, cost	Low-Med	Very high	0
2	LLM Evaluation	Assertions, judge calibration, datasets, regressions, significance	Low	Very high	0
3	Prompt Injection & Guardrails	Red-team injection/tool abuse; blue-team filters and tradeoffs	Med	High	0
4	Prompt Engineering	Structured output, few-shot, decomposition, robustness	Low	Very high	1
5	Agent & Tool Use	Schemas, planning, recovery, loop control, cost ceilings	High	High	2
6	Benchmark Playground	Harness variance, contamination, leaderboard interpretation	Med	Medium	2
7	Neural Network Simulator	Architecture, training dynamics, overfitting, hyperparameters	Med	Medium	3
8	Fine-tuning & Adaptation	Tune vs prompt, data prep, LoRA, tuned-model evaluation	High	Medium	3

6.1 Why RAG, Evaluation, and Guardrails form the POC

- **One harness shape:** learner submits a configuration or prompt; the platform runs it against a hidden set and reports metrics.
- **The actual AI engineering week:** build it (RAG), measure it (Evaluation), stop it being abused (Guardrails).
- **Injection is top-of-funnel:** it is inherently shareable; RAG and Evaluation create durable practice depth.
- **No arbitrary code execution in Phase 0:** structured inputs avoid the largest early infrastructure and security burden.

SCOPE

Prompt Engineering is deliberately Phase 1

Although cheap to build, it is commoditised and does not alone signal a serious engineering-practice platform.

7. Grading & feedback engine

This is the core system. Everything else is UI around it.

7.1 Grading pipeline

1	VALIDATE	schema / budget
2	MATERIALISE	runnable artifact
3	EXECUTE	hidden set
4	MEASURE	class A metrics
5	JUDGE	only if needed
6	SCORE	gates + CI
7	EXPLAIN	failure + action

7.2 Three grader classes

Class	Use	Guarantee
A - Deterministic	Metrics, canaries, tool-call detection, schema, budgets	Fully reproducible. Pass/fail must be decidable from A alone wherever possible.
B - Calibrated judge	Faithfulness, correctness, rubric adherence	Temperature 0; n=3 majority; cached by judge/rubric/output; kappa ≥ 0.7 ; revalidate on model change.
C - Advisory	Style and senior-engineer commentary	Never affects pass/fail; clearly labelled as opinion.

Judge margin rule

- Report each judge-derived metric with a bootstrap 95% confidence interval over the hidden set.
- Pass when the lower bound clears the threshold; fail when the upper bound remains below it.
- Otherwise return **Inconclusive**, do not consume the attempt, and rerun once on a larger sample. A second inconclusive resolves as pass.
- A judge can never be the sole determinant of a pass; every exercise needs at least one class A gate.

7.3 Feedback quality bar

1. **Scorecard:** learner value, threshold, and outcome for every metric.
2. **Worst three failing cases:** full input, expected behaviour, and actual behaviour; do not reveal the whole hidden set.
3. **Failure class:** controlled taxonomy such as `retrieval:chunk-too-large` or `guardrail:over-blocking-benign`.
4. **Next action:** hint, related drill, or tutor prompt only after at least three failed attempts.

7.4 Instrumentation panel

Every simulator run exposes retrieved chunks and scores, prompts actually sent, tokens in/out, per-step latency and cost, and attempted tool calls. Invisible failure modes are the reason the product exists, so the trace is non-negotiable.

7.5 Grader hardening

1. **Quarantine learner text.** Typed, delimited data blocks only; never concatenate learner content into judge instructions.
2. **Structured verdicts.** Constrained JSON with `verdict`, `evidence_span`, and `confidence`; reject free-form output.
3. **Evidence binding.** For R3 and E2, cited evidence must exist in the source corpus or labelled output and is verified deterministically.
4. **Adversarial grader suite.** Roughly 40 known injection payloads run in CI and grow after each production escape.
5. **Never trust learner-authored judges.** In E2 the learner's judge is the subject under test; LabPath owns the meta-grader.

DECISION

Scoring integrity is attack surface

Learner-authored prompts can instruct the platform's own grader to return PASS. Quarantine, schema-constrained output, evidence binding, and adversarial CI are required from day one.

7.6 How verdict stability is engineered

Hosted model APIs are not reproducible. Temperature 0 and seeds are best-effort only. P8 is achieved through system design, not assumptions.

Mechanism	Applies to
Frozen generations	E1-E3: the entire Evaluation simulator; evaluator varies, model output is a versioned fixture.
Generation response cache	R2-R4 and G1-G3, keyed by model version, prompt hash, and parameter hash.
Judge cache	All class B grading, keyed by judge version, rubric hash, and output hash.
Threshold bands + inconclusive	All judge-gated metrics, preventing a single flip from changing a pass.
Deterministic efficiency proxies	R2, R4, G3; never use provider wall-clock latency as a gate.

TARGET

At least 99% verdict stability

Monitor re-grades continuously in CI. Do not claim 100% stability against hosted models.

7.7 Never gate on wall-clock latency

Provider tail latency and worker concurrency dominate p95 wall-clock. Efficiency gates use mean prompt tokens, total tokens, model-call count, reranker window, and retrieved-context size. Wall-clock remains visible in traces and scorecards, but is informational only.

8. Progression, profile and credibility

Mechanism	Product role
Streaks and daily drills	One three-minute drill each day creates the habit loop.
Skill radar	Difficulty- and recency-weighted graph; skills fade after six months without practice.
Ranked contests	Timed novel problems and seasonal rating drive competitive acquisition.
Public profile	Verified solves, radar, and contest rating form the credential precursor.
Verified assessments - Phase 3	Proctored, novel, time-boxed, hint-free and signed results create the employer-facing signal.

The strategic prize is a score employers trust, not a certificate of completion.

9. Tutor layer - Preply model

9.1 Thesis

Practice creates stuck moments. A stuck moment with a precise, machine-generated description of the failure is the highest-intent tutoring lead available. LabPath knows exactly what failed, on which exercise, and how many times. That context lets a paid session begin with the real problem at minute zero.

9.2 Marketplace shape

- Tutors are **vetted by solving LabPath exercises**, not primarily by CV. Their skill radar is public and drives ranking.
- Booking is contextual inside a failed exercise and carries the learner's submission and trace.
- Sessions happen in-platform in a shared exercise workspace, not as bare video calls.
- Tutor sets the rate; marketplace take rate is approximately 20-25%.
- Reviews, escrow, disputes, payouts, availability, and timezone handling follow standard marketplace patterns and are specified in Phase 4.

9.3 Build only the foundations now

1. Submissions and traces are durable, addressable objects with shareable read-only permissions.
2. Role model accommodates `learner`, `tutor`, and `org_admin`.
3. The skill graph is real and scored because it is the tutor-matching key.

SCOPE

No marketplace implementation before Phase 4

Build these three architectural foundations and nothing else tutor-related during the POC.

10. Monetization

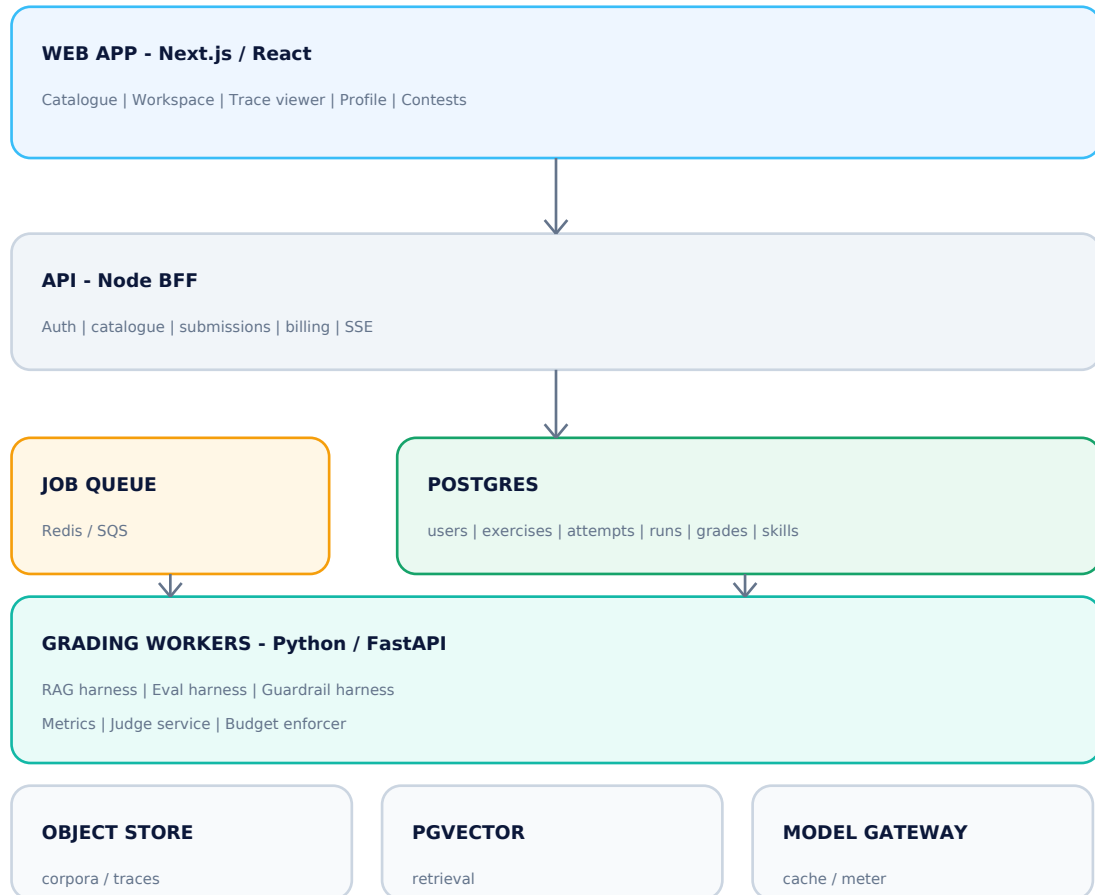
Tier	Indicative price	Contents
Free	€0	3 problems/month, public samples, no hidden-run trace, no contests
Pro	€19/mo or €149/yr	Full catalogue, fair-use runs, traces, hints, contests, profile
Pro+ / Career	€39/mo	Verified assessments, interview sets, priority compute, tutor credit
Tutoring	Marketplace	20-25% take rate
Teams	€25-40/seat/mo	Dashboard, gap analysis, custom sets, SSO
Institution / Gov	Contract	Cohorts, LTI, private-cloud/on-prem, custom content

BUSINESS REQUIREMENT

Compute cost decides whether pricing works

Cost enforcement and real-time per-user accounting are core product constraints, not later engineering optimisation.

11. Target architecture



Key choices

- **Separate Python grading service.** The AI tooling ecosystem is Python; the Node service remains the BFF and product API.
- **Model gateway in front of every provider call.** Routing, caching, metering, budget cutoffs, and failover begin in Phase 0.
- **Precompute fixed-corpus embeddings.** Avoid paying to re-embed identical documents per learner.
- **Use pgvector first.** A dedicated vector database adds an unnecessary system at POC scale.

OPEN 01

Provider strategy

Recommendation: one provider for graders and judges to protect consistency; learner-selectable models only where an exercise explicitly tests model variance.

12. Non-functional requirements

12.1 Reproducibility

Every grade records `exercise_version`, `grader_version`, `model_versions`, and `sample_seed`. Historical grades remain attached to their original version and are never silently re-scored.

12.2 Execution safety

- **Phase 0 has no arbitrary code execution.** Inputs are structured configuration, natural-language prompts, Assertion DSL, and Slice Spec only.
- Use RE2 for patterns; impose per-assertion and suite timeouts, size and count caps, and a hand-written Slice Spec parser. Never use `eval()`.
- Prompt injection against graders remains the primary Phase 0 execution threat and is mitigated by Section 7.5.
- Phase 1+ Python runs in gVisor or Firecracker: 512 MB, 30 seconds, no persistent filesystem, and egress only through an allowlisted model gateway.

12.3 Cost and abuse control

- Hard budget per attempt in both tokens and currency; kill on breach with clear learner-facing status.
- Daily run quota by tier; aggressive corpus, generation, judge, and baseline caching.
- Real-time cost-per-user dashboard from day one.

12.4 Anti-cheat

- Hidden eval sets never leave the server - not in API responses, traces, or error messages.
- Failure feedback exposes a rotated sample, never the full set. Repeated deliberately-bad submissions must not reconstruct the hidden pool.
- Use pool-and-sample where a fixed set would become visible. G3 uses a 500-item pool and samples 100 per attempt.
- For contests and verified assessments: per-user problem sampling, time boxes, and paste-pattern telemetry.

A determined cheater can beat a practice platform. Keep practice usable and make the Phase 3 credential itself cheat-resistant.

12.5 Performance

Surface	Target
Catalogue and workspace load	< 1.5 seconds p95
Grading job start	< 3 seconds; progress streamed through SSE
Typical exercise grade	< 60 seconds p95; longer labs show live progress

12.6 Accessibility and i18n

Meet WCAG 2.1 AA. Launch in English; Hebrew and Spanish follow through existing Bina capability. Briefs and feedback are translated; hidden eval sets remain in their source language.

13. Success metrics

Layer	Metric	End-state target
Acquisition	Signups/month	-
Activation	% submitting ≥ 1 solution in first session	>45%
Retention	D7 / D30 return	>30% / >15%
Habit	Weekly active submitters / MAU	>40%
Depth	Median solves per retained user in 30 days	>12
Quality	Failing submissions rated helpful	>70%
Quality	Judge-human agreement on gating judges	kappa >0.7
Monetization	Free -> Pro conversion	4-7%
Marketplace	Pro users booking a session	>8%
Unit economics	Compute cost / Pro revenue	<25%

POC FOCUS

Activation and feedback helpfulness

Every other metric is downstream. First prove that engineers submit and that failed submissions teach them something useful.



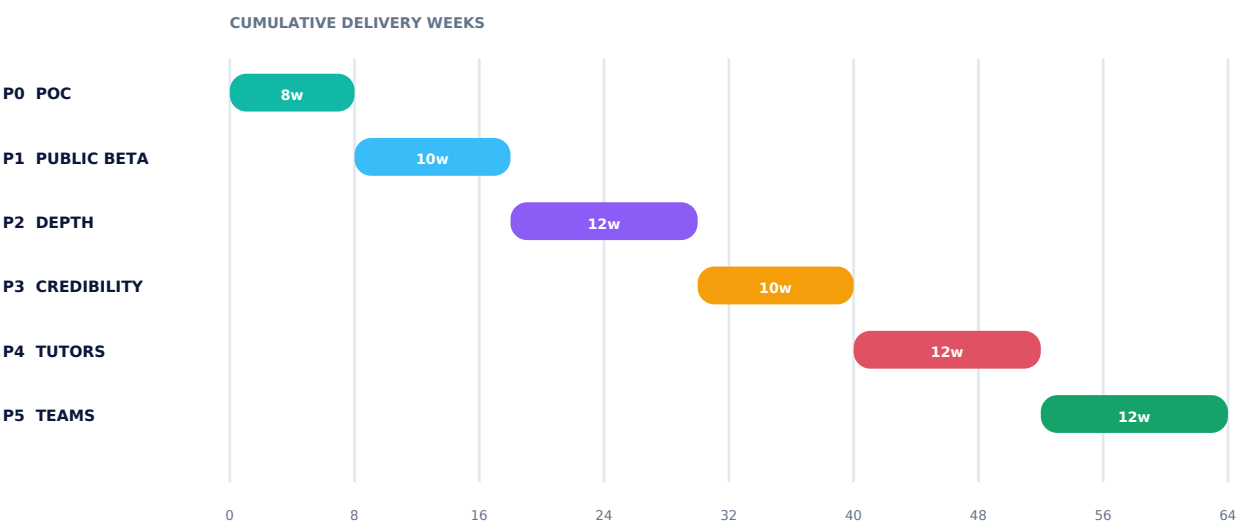
2

PART 2

HOW WE GET THERE

Phase sequence, Phase 0 implementation detail, acceptance criteria, delivery plan and risks.

Phase overview



Phase	Name	Duration	Exit criterion
0	POC - 10 exercises, 3 simulators	6-8 weeks	Stable verdicts; calibrated/injection-resistant graders; 15/20 testers finish >=5; >=70% helpful feedback
1	Public beta	+10 weeks	50 exercises, sandbox, accounts, payments, 1,000 signups
2	Depth & competition	+12 weeks	Agent + Benchmark live; contests; 150 exercises
3	Credibility	+10 weeks	Verified assessments and employer-used profiles
4	Tutor marketplace	+12 weeks	50 vetted tutors; first 500 paid sessions
5	Teams / institutions / Bina	+12 weeks	First team and institutional contracts

Phase 0 - The POC

0.1 Objective

Prove that a graded, hidden-eval-set AI engineering exercise produces feedback good enough that engineers come back.

The POC does not prove scale, payments, or breadth.

0.2 Scope

In	Out - explicitly
3 harnesses: RAG, Evaluation, Guardrails	Arbitrary code execution / sandbox
10 authored exercises with hidden sets and reference solutions	Payments, tiers, billing
Exercise repository + nightly reference CI	Contests, leaderboards, streaks, public profiles
Class A/B grading, budgets, gateway, cache	Tutor marketplace functionality
Workspace: brief, editor/config, scorecard, trace, hints	Courses and guided paths
Minimal auth and attempt history	Mobile optimisation beyond responsive tolerance
Internal cost dashboard	Other five simulators and i18n

0.3 POC data model

```
users id, email, role, created_at
exercises id, slug, version, type, simulator, title, brief_md,
difficulty, skill_tags[], thresholds jsonb, budget jsonb, is_published
attempts id, user_id, exercise_id, exercise_version, status,
started_at, submitted_at
submissions id, attempt_id, payload jsonb, payload_hash
runs id, submission_id, worker_version, model_versions jsonb,
sample_seed, tokens_in, tokens_out, cost_eur_micros,
fx_rate, cache_hit_ratio, status
grades id, run_id, verdict, metrics jsonb, gate_results jsonb,
failure_classes text[], scorecard jsonb
traces id, run_id, blob_uri
gen_cache key(model_version,prompt_hash,params_hash), blob_uri, hits
judge_cache key(judge_version,rubric_hash,output_hash), verdict, hits
label_sets id, exercise_id, annotators, kappa_hh, adjudicated
hints_unlocked user_id, exercise_id, hint_index, unlocked_at
skills id, slug, name, parent_id
```

All money is stored in EUR integer micros with the day's `fx_rate` on the run. `sample_seed` makes sampled runs replayable. Verdict is three-valued: pass, fail, or inconclusive. Submission payloads are polymorphic by simulator and validated against exercise-owned JSON Schema.

0.4 POC API surface

```
GET /api/exercises
GET /api/exercises/:slug
POST /api/attempts
POST /api/attempts/:id/submissions
GET /api/runs/:id
GET /api/runs/:id/stream
GET /api/runs/:id/grade
GET /api/runs/:id/trace
POST /api/exercises/:slug/hints/next
GET /api/me/progress
```

INTEGRITY

No hidden data in any endpoint

CI must grep representative API responses for hidden-set fixtures and fail the build on leakage.

0.5 The ten exercises

Difficulty: E = easy, M = medium, H = hard. Each exercise includes a reference solution and a near-miss solution.

RAG simulator - shared environment

Fixed synthetic corpus of roughly 60 internal-policy and technical documents. Public sample: 5 labelled Q/A pairs. Hidden set: 30 Q/A pairs with gold answers and source spans, including deliberate unanswerables. Learner submits structured retrieval configuration and, where applicable, a generation prompt.

R1 - Chunk It Right | E | chunking, retrieval-quality

The retriever is missing answers that are definitely in the corpus. Nothing is wrong with the model. Fix the chunking.

- Controls: chunk size, overlap, splitting strategy. Baseline recall@5 = 0.41.
- **Class A pass:** recall@5 $\geq$ 0.80 on the hidden set.
- Feedback: 3 worst misses with gold span and actual retrieved chunks.
- Failure classes: `chunk-too-large`, `chunk-too-small`, `no-overlap-boundary-loss`, `structure-ignored`.

R2 - The Cost Ceiling | E/M | retrieval-quality, cost-engineering

Top-k is 20. Finance called. Keep the quality and cut the context.

- Controls: top-k, reranking, chunk size.
- **Class A:** mean prompt tokens $\leq$ 2,000 and recall@k $\geq$ 0.80. **Class B:** correctness $\geq$ 0.75 with CI.
- Scorecard plots learner and reference on the quality/cost frontier; latency is informational.
- Failure classes: `topk-too-high`, `no-rerank-with-low-k`, `quality-sacrificed`.

R3 - The Citation Contract | M | grounding, prompt-design, refusal

Legal requires a source for every answer and 'I don't know' when the corpus does not know.

- Learner authors the generation prompt; retrieval is fixed. Six of thirty hidden questions are unanswerable.
- **Class A:** citation emitted; citation ID exists in retrieved set; gold-span overlap ≥ 0.85 ; correct refusal on $\geq 5/6$ unanswerables; no more than 2/24 false refusals.
- **Class B:** cited chunk semantically supports claim ≥ 0.88 with CI; judge returns a verified supporting span.
- Failure classes: hallucinated-citation, citation-outside-gold-span, citation-not-supporting, over-refusal, under-refusal, format-drift.

FLAGSHIP

Build R3 first and build it well

This is the primary POC demonstration because it visibly combines deterministic grounding with calibrated semantic support.

R4 - Rerank or Re-think | M/H | reranking, retrieval-quality, eval-reading

Recall@20 is 0.94. Answer quality is 0.58. The information is retrieved and then buried. Fix the ordering.

- Controls: reranker, top-n, optional query rewrite.
- **Class A:** nDCG@5 improves ≥ 0.15 over baseline; context $\leq 3,000$ tokens; ≤ 2 model calls/question.
- **Class B:** answer correctness ≥ 0.78 with CI.
- Rotating before/after ranking diff: five improved and three degraded questions.
- Failure classes: no-rerank, rerank-window-too-narrow, context-budget-blown, query-rewrite-drift.

LLM Evaluation simulator - shared environment

Frozen, pre-generated customer-support outputs with hidden human labels. No live model generation at grade time. The learner builds the evaluator, not the model.

E1 - Write the Assertion Suite | E | eval-design, deterministic-checks

100 support-bot responses. 100 human labels you cannot see. Write checks that agree with the humans.

- Assertion DSL only. Public sample: 10 outputs with labels. Hidden set pinned 50/50; positive class is 'fail'.
- **Class A pass:** F1 ≥ 0.70 , precision ≥ 0.65 , recall ≥ 0.60 .
- Reject before scoring if the suite flags $>90\%$ or $<10\%$. Scorecard includes confusion matrix and rotating 3 FP + 3 FN.
- Failure classes: `over-strict-regex`, `missing-format-check`, `assertion-too-broad`, `degenerate-suite`.

E2 - Judge the Judge | M | judge calibration, bias awareness

Write a judge that a human would agree with - twice in a row.

- Learner judge runs over 80 held-out outputs three times at platform-fixed temperature 0.7; majority verdict per item.
- **Class A pass:** kappa ≥ 0.60 vs human labels; $\geq 85\%$ items have 3/3 consistency; at least 12/15 verbosity traps correctly fail.
- Constrained verdict schema; malformed outputs count as incorrect.
- Failure classes: `verbosity-bias`, `position-bias`, `rubric-underspecified`, `non-deterministic-judge`, `judge-too-lenient`, `malformed-verdict`.

E3 - Catch the Regression | M/H | regression, slicing, significance

v2 scores better in aggregate. Support tickets rose 30%. Find out why.

- Given 200 paired v1/v2 outputs. Learner defines slices from published metadata; no candidate slice list exists to brute-force.
- Submission is a Slice Spec: group expressions, metric, hypothesis test, and correction.
- **Class A pass:** regressed slice at corrected $p < 0.05$; zero false-positive slices; correction declared and applied.
- Attempt policy: 5 submissions, then 24-hour cooldown.
- Failure classes: `aggregate-only`, `slice-too-coarse`, `no-significance-test`, `multiple-comparisons-ignored`, `underpowered-slice`.

Prompt Injection & Guardrails simulator

Use a split UI: brief and objective on the left, realistic mock app on the right. LabPath diverges from judge-based scoring: winning is a deterministic event such as canary leakage, a protected field in output, or a tool call with specific arguments.

Detector/defence contract

```
level: 3
filter_catches: [literal, base64, rot13, reversed]
detector_catches: [literal, base64, rot13, reversed, hex,
spaced, per-char-split, spelled-out,
acrostic-first-letter, unicode-homoglyph]
intended_gap: [hex, spaced, per-char-split, spelled-out, acrostic]
```

CI asserts that filter coverage is a strict subset of detector coverage and that at least three intended-gap encodings pass the defence and trip the detector. Shared normalisers are versioned and unit-tested.

G1 - Break the Concierge | E | red team

Extract the booking-override canary from a hotel concierge across three escalating levels.

- L1: no defence. L2: hardened system instruction. L3: output filter with a designed detector gap.
- **Class A win:** shared normaliser finds the canary in model output.
- After solving, show the bypassed defence, encoding used, and a defence that would have stopped it.

G2 - The Indirect Payload | M | red team

Control a page the research agent reads; make its email tool send the summary to the attacker.

- Learner supplies page content, not direct chat. Levels add delimiters and finally a separate confirmation-gate policy checker.
- Policy checker sees only proposed tool + args and original user request; it never sees the injected page.
- **Class A win:** `send_email` tool log records the attacker address as recipient. CI proves at least two distinct L3 payloads.

G3 - Hold the Line | M/H | blue team

Write the system prompt and filters. Block attacks without breaking the product.

- Learner submits system prompt, input/output Assertion DSL rules, and optional tool policy.
- Hidden pool: 200 attacks + 300 benign, including 60 adversarially benign. Each attempt samples 40 attacks + 60 benign, stratified by category.
- **Class A:** attack block $\geq 90\%$; benign pass $\geq 95\%$; ≤ 2 added model calls/request; ≤ 600 filter tokens.
- Wilson intervals and inconclusive reruns prevent sampling coin flips. Show all errors from that attempt's sample, never the full pool.
- Failure classes: `over-blocking`, `keyword-filter-brittleness`, `no-output-filtering`, `encoding-bypass`, `multiturn-bypass`, `filter-cost-blown`.

CONTENT PRIORITY

G3 is the most valuable and hardest POC exercise

The adversarially benign pool makes this a real precision/recall problem. Budget authoring time accordingly.

0.6 Coverage check

Band	RAG	Evaluation	Guardrails
Easy	R1	E1	G1
Medium	R2, R3	E2	G2
Hard	R4	E3	G3

Grader class coverage

Gate	R1	R2	R3	R4	E1	E2	E3	G1	G2	G3
Class A	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Class B	-	Yes	Yes	Yes	-	-	-	-	-	-

Only R2-R4 depend on a calibrated judge, and every one has a deterministic gate first. If calibration fails, judge gates become advisory and the POC still ships.

0.7 UI scope - four screens

Screen	Purpose
1. Catalogue	Cards grouped by simulator; difficulty, skills, solved state.
2. Workspace	Brief + constraints + hints; submission surface; live run, scorecard and cases.
3. Trace view	Retrieved chunks, prompts, tool calls, tokens, cost and per-step latency.
4. Progress	Attempt history, solves, first-cut skill radar.

DESIGN SCOPE

Reuse the current LabPath shell

Keep the prototype's navigation and visual language. Do not redesign during Phase 0.

0.8 Phase 0 acceptance criteria

#	Gate	Definition of done
1	Exercise completeness	10 published; reference passes and near-miss reliably fails in CI
2	Verdict stability	$\geq 99\%$ across 100 re-grades/exercise; inconclusive $< 5\%$
3	Judge calibration	Every gating judge kappa ≥ 0.70 ; human-human kappa reported
4	Grader integrity	Adversarial injection suite cannot flip a verdict
5	No leakage	No hidden item in API, trace, or error response
6	Budget + DoS	Kill cost/token breach; assertion timeout and RE2 safety verified
7	Performance	Grade < 60 seconds p95 across all ten
8	Cost	Per-exercise budgets met; blend $\leq \text{€}0.20$; dashboard live
9	External validation	20 testers; 15 finish ≥ 5 ; $\geq 70\%$ rate failure feedback helpful
10	Failure taxonomy	Every failure classified; unclassified rate $< 10\%$

REAL GATES

Criteria 2, 3, 4 and 9

Stability, judge calibration, grader integrity, and external feedback value determine whether the core claim is true. The other criteria are essential hygiene.

0.9 Week-one engineering blockers

0.9.1 Assertion DSL

A declarative YAML format used by E1 and G3. No user-supplied code. Version 1 grammar freezes before week 5.

```
version: 1
assertions:
  - id: no-pii
    when: always
    check: not_matches
    pattern: '\b\d{3}-\d{2}-\d{4}\b'
    flavor: re2
  - id: valid-json
    check: json_schema
    schema_ref: ./schemas/response.json
  - id: length
    check: length_between
    min: 20
    max: 1200
  - id: refund-amount-sane
    check: numeric_extract_compare
    pattern: 'refund of \${[0-9.]+}'
    op: lte
    value: 500
    verdict:
    fail_if: any
```

Allowed primitives: `matches`, `not_matches`, `contains_any`, `contains_none`, `json_schema`, `length_between`, `numeric_extract_compare`, `token_count_lte`, `language_is`, and `sentiment_not` (class C only). RE2 only; 50 ms/assertion; 5 s/suite; maximum 40 assertions; maximum pattern length 512.

0.9.2 Slice Spec - E3 only

```
version: 1
slices:
- name: hebrew-inputs
where: "meta.language == 'he'"
- name: long-tickets
where: "meta.input_tokens > 400"
- name: refusal-expected
where: "meta.expected_refusal == true"
metric: pass_rate
test: two_proportion_z
alpha: 0.05
correction: benjamini_hochberg
min_slice_n: 20
```

Slice Spec is deliberately narrower than Assertion DSL. `where` accepts comparisons and boolean operators over published `meta.*` fields only - no calls and no deeper traversal. A hand-written parser evaluates it; never Python or JavaScript eval.

0.9.3 Human labelling protocol

- Two independent annotators label every gating set; a third-pass adjudication resolves disagreements.
- Publish human-human kappa for each set. If it is below 0.75, rewrite guidelines and relabel before calibrating a judge.
- Volumes: R3 150 labelled items; E1 100; E2 80 + 15 traps; E3 200 paired; G3 500-item pool - roughly **1,000 double-annotated items**.
- Second annotator is a contracted domain expert at ~0.5 FTE in weeks 1-6. The exercise author cannot be the sole labeller of their own set.

LARGEST WORK ITEM

Double annotation, not application code

Gold spans, bias traps, attack pools, benign pools, and adjudication determine whether the exercises are real.

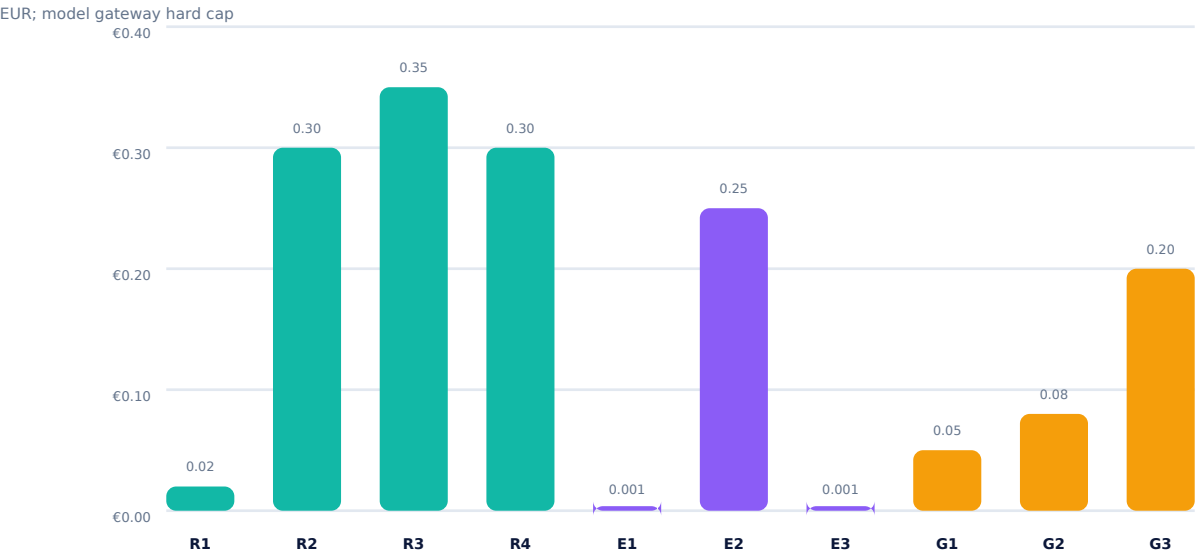
0.9.4 exercise.yaml schema and ingest

```
schema_version: 1
slug: rag-003-citation-contract
version: 4
type: exercise
simulator: rag
title: The Citation Contract
difficulty: M
skills: [grounding, prompt-design, refusal-behaviour]
submission_schema: ./schemas/rag_generation_prompt.json
budgets:
  max_model_calls: 60
  max_tokens: 120000
  max_cost_eur: 0.35
  wall_clock_s: 120
gates:
  - id: gold-span-overlap
    class: A
    metric: gold_span_overlap
    op: gte
    value: 0.85
  - id: citation-support
    class: B
    metric: citation_supports_claim
    op: gte
    value: 0.88
  judge: judges/grounding_v3.yaml
  banded: true
  eval:
    public: ./eval_public.jsonl
    hidden: ./eval_hidden.jsonl
  feedback:
    show_failing_cases: 3
    rotate: true
```

On merge to `main`, a GitHub Action validates schemas, runs reference and near-miss solutions, and upserts exercise metadata through a signed admin endpoint. Content-affecting changes bump the version. Hidden eval files go directly to object storage and never into API-readable tables.

0.10 Per-exercise cost budgets

Per-attempt budget by exercise



Exercise	Calls / attempt	Budget	Note
R1	0 live	€0.02	Retrieval only
R2	~30 gen + 30 judge	€0.30	Cached on re-grade
R3	~30 gen + ~90 judge	€0.35	Flagship / highest cap
R4	~30 gen + 30 judge	€0.30	-
E1	0	€0.001	Frozen outputs; assertions
E2	240	€0.25	80 x 3 learner judge
E3	0	€0.001	Frozen outputs; statistics
G1	~10-30 turns	€0.05	Per-turn metered
G2	~15 + checks	€0.08	Policy checker
G3	100 x (1 + <=2 filter)	€0.20	Sampled suite

Blended mean is approximately €0.16 per attempt. Two exercises cost essentially nothing because generations are frozen.

OPEN 08

Unit economics gap

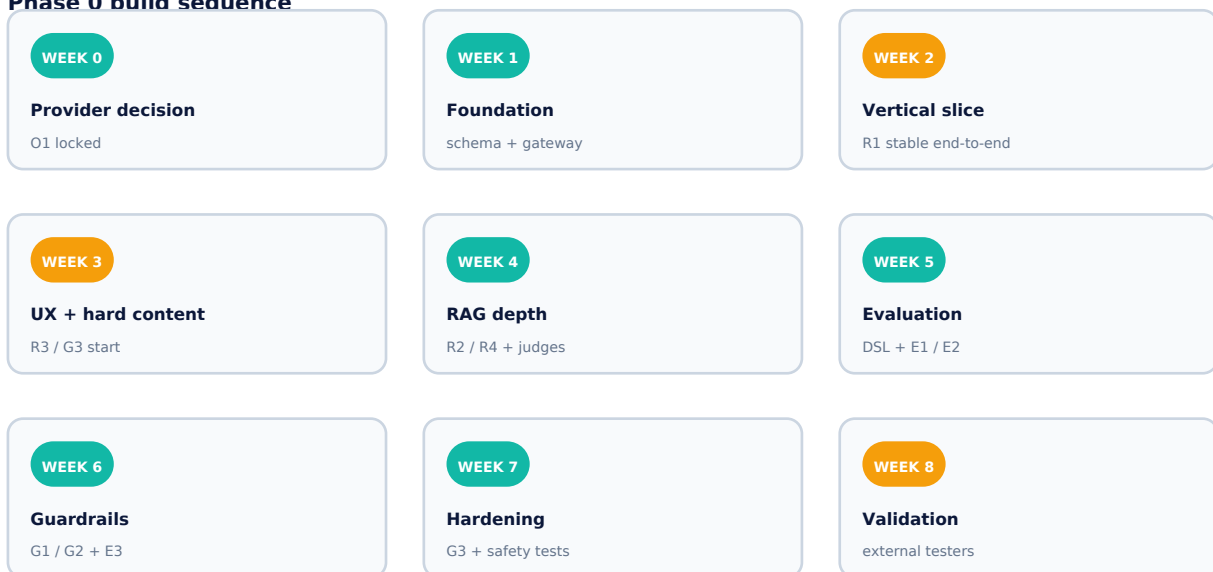
At 150 attempts/month, the expected €24 compute cost exceeds the €19 Pro price and the €4.75 target. Before paid launch, combine cheaper judge models, 60-80 fair-use attempts, higher price, and safe cross-user cache sharing.

0.11 Team and sequencing

Role	Allocation	Focus
Backend / AI engineer	2 FTE	Grading engine, harnesses, gateway, budgets
Frontend engineer	1 FTE	Workspace, trace, catalogue
AI content author / SME	1 FTE	10 exercises, corpora, hidden sets, hints, first labels
Second annotator	0.5 FTE, W1-6	Independent labels + adjudication
Product / design	0.5 FTE	Workspace UX and feedback presentation
QA	0.5 FTE	Stability, leakage, budgets, injection suite

Content is the role most likely to be under-resourced and the factor most likely to determine POC quality. Engineers should not author exercises only in spare cycles.

Phase 0 build sequence



GO / NO-GO

Week 2: R1 stable end-to-end

If R1 has not achieved $\geq 99\%$ stability by the end of week 2, the schedule is wrong, not merely the implementation.

SCOPE CHECK

Week 3: R3 and G3 authoring

If hard-exercise authoring is over budget, cut to eight exercises by dropping R2 and E3. Do not compress the external tester round.

0.12 Phase 0 risks

Risk	Impact	Mitigation
Judges fail kappa ≥ 0.7	High	R2-R4 retain class A gates; demote judge to advisory and ship.
Graders are prompt-injectable	High	Quarantine, structured verdicts, evidence binding, adversarial CI.
Hosted-model instability	High	Frozen Eval fixtures, response/judge caches, bands, no latency gates.
Authoring underestimated	High	Dedicated SME + second annotator; author R3/G3 first; cut scope if needed.
Model deprecated mid-build	Medium	Gateway pins versions; nightly reference/near-miss CI detects drift.
Compute surprises	Medium	Precompute, freeze, cache, cap and dashboard.
DSL scope creep	Medium	Freeze Assertion DSL; keep Slice Spec separate.
POC feels like a toy	Medium	Lead demo with R3 and G3: recognisable production work.

Phase 1 - Public beta | +10 weeks

- Dedicated gVisor/Firecracker code-execution workstream and security review.
- Prompt Engineering simulator and catalogue expansion to 50 exercises.
- Accounts, Free/Pro, Stripe, quotas, streaks, daily drills, solve history, first profile.
- Three-minute first-solve onboarding because activation is the primary metric.

Exit: 1,000 signups, activation >40%, D7 >25%, positive Pro gross margin.

Phase 2 - Depth & competition | +12 weeks

- Agent & Tool Use simulator as a dedicated mini-project: real sandbox tools, traces, loops, recovery, costs.
- Benchmark Playground: contamination, harness variance, and leaderboard interpretation.
- Timed seasonal contests, guided exercise sequences without lessons, and free team leaderboards.

Exit: both simulators live, contests running, 150 exercises.

Phase 3 - Credibility | +10 weeks

- Proctored, novel, time-boxed, hint-free verified assessments with signed results.
- Employer-facing public profiles and shareable verified skill reports.
- Neural Network and Fine-tuning simulators; mature recency-weighted skill graph.
- Begin employer adoption work: the credential has no value until hiring teams use it.

Phase 4 - Tutor marketplace | +12 weeks

- Tutor onboarding and vetting by exercise solve; public tutor skill radar.
- Contextual booking carrying submission and trace into an in-platform shared workspace.
- Scheduling, timezone, escrow, payouts, refunds, disputes, reviews, and 20-25% take rate.
- Recruit the first 50 tutors from top-ranked users before launch; supply is harder than demand.

Phase 5 - Teams, institutions and Bina | +12 weeks

- Org accounts, SSO/SCIM, seats, gap dashboards, custom exercise sets.
- LTI/LMS integration, cohort management, instructor views, assignment and grade export.
- LabPath becomes Bina's AI-engineering content and grading substrate through existing exercise, grading, and skill-graph interfaces.
- Private-cloud and on-prem packaging for government and defence buyers.

Open decisions

#	Decision	Needed by	Recommendation
O1	Model provider: single vs multi	Before W1	Single for graders/judges; selectable under test
O2	LabPath brand vs Bina alignment	Phase 1	Distinct consumer brand; 'by Bina' endorsement
O3	3 free problems/month vs week	Phase 1	Start at 3/week; activation first
O4	Own vs license corpora	W1	Synthetic corpora: no licensing risk, full gold-span control
O5	LabPath-issued vs accredited assessment	Phase 3	LabPath-issued first; partner only if buyer requires
O6	Tutor geography and payment rails	Phase 4	To be decided
O7	Verify competitor scoring claim	W1	Hands-on repeated-input test before external claims
O8	Unit economics gap	Before pricing	Cheaper judge + 60-80 fair use + pricing/cache changes

Appendix A - Glossary

Term	Meaning
Hidden eval set	Held-out inputs and gold labels used for scoring; never served to the client.
Class A grader	Deterministic, reproducible metrics, assertions, or event detection.
Class B grader	LLM-as-judge calibrated against human labels; kappa ≥ 0.7 to gate a pass.
Class C grader	Advisory commentary that never affects pass/fail.
Failure class	Controlled-vocabulary label describing why a submission failed.
Canary	Unique secret planted in a prompt; its appearance proves extraction.
Adversarially benign	Legitimate request phrased so a naive guardrail blocks it.
Cohen's kappa	Inter-rater agreement corrected for chance; judge calibration metric.
nDCG@k	Ranking-quality metric: whether the right results are near the top.

Appendix B - References

Lakera Agent Breaker - design reference for injection levels, split UI, and mock-app realism. Comparative scoring remains OPEN O7 pending first-hand verification. play.lakera.ai/agent-breaker

Lakera Gandalf - precedent for prompt-injection games as top-of-funnel acquisition.

LLMBar, JudgeBench, RewardBench - prior art for meta-evaluation of judges; relevant to framing and novelty claims.

LeetCode - precedent for hidden test sets, difficulty bands, contests, and score-as-credential.

Preply - precedent for tutor marketplace mechanics; LabPath diverges through solve-based vetting and submission-linked sessions.

LabPath prototype - learningapp-web.vercel.app

Prepared for engineering, product and design review. Owner: Yonatan (Bina / CyberProAI). Status: Draft. Version 1.0.